

AUTO ESPÍA

Grupo: Aquino Tomás,

Borcard Jonathan,

[Chambi Carlos.](#)

Mamani Limber,

Morales Leonel.

Materia: Laboratorio de mediciones y ensayos III;

Procesamiento de imagen y sonido;

Sistemas electrónicos embebidos;

Sistemas de control de procesos

Redes y sistemas de comunicación.

Docente: Palmese Sebastian

Curso: 6to1ra ELECTRÓNICA

links:

▶ Surveillance Car using ESP32 Cam module | ESP32 Camera wi-fi car 🚗

▶ ESP32 CAM Surveillance Car | SPY Camera Car

▶ Tutorial ESP32 CAM Rover

▶ Advanced Mecanum Wheels Robot using Arduino & ESP32 CAM || 4WD Omnidirectio...

▶ Building a SPY CAR at Home | Robot Car with Camera | DIY Projects | The Wrench

▶ ESP32 based omnidirectional robots w/ camera | makermoeke

[Coche robot 2WD Arduino- Programación L298N \(luisllamas.es\)](#)

links código para esp 32 cam:

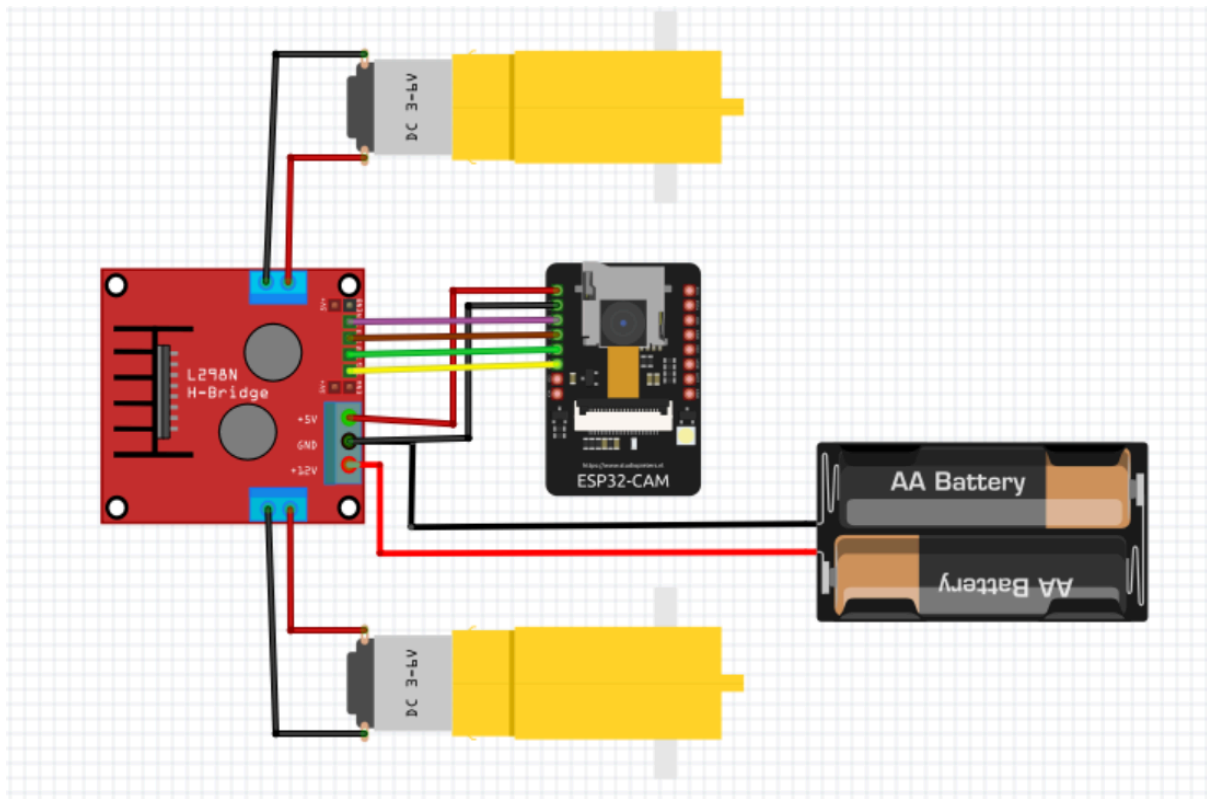
<https://hetpro-store.com/TUTORIALES/esp32-cam-como-servidor-web-con-camara/>

<https://wiki.eurek.org/codificacion-y-decodificacion-del-ejemplo-camerawebserver/>

http://kio4.com/arduino/247_Wemos_WebCam.htm

<https://www.youtube.com/watch?v=1tohZ9W1kRQ>

ESQUEMA:



LISTA DE COMPONENTES:

- Esp 32 Cam
- L298n controlador de motores
- 3 baterías de 3,7v de litio 18650 recargables
- 2 motorreductores con ruedas
- 1 rueda loca giratoria
- 1 switch deslizante
- 1 porta baterias triple de 18650

PRINCIPIO DE FUNCIONAMIENTO:

Este proyecto funciona como un auto controlado por un esp32 cam, siendo el único microcontrolador y con su respectiva programación tener un control total del prototipo para avanzar en diferentes direcciones desde una página web y teniendo como función una cámara que transmite todo lo que se puede visualizar desde el dispositivo en la página web en un recuadro, esta conexión a la página web se obtiene usando el ip del wifi del mismo microcontrolador esp32 cam que tiene conexión a wifi y bluetooth incluidos.

Este proyecto implica el uso de un ESP32-CAM como único controlador para un automóvil. El ESP32-CAM se conecta a una red WiFi y ofrece una interfaz de control a través de una página web. Desde esta página, los usuarios pueden enviar comandos para controlar la dirección del automóvil. Además, el proyecto incluye una cámara que transmite video en vivo desde el automóvil a la misma página web, lo que permite a los usuarios ver lo que ve el automóvil mientras lo controlan.

CÁLCULO DE SUPERFICIE:

L298n Puente H	43mm x 43mm	1849 mm2
Esp 32 Cam	27mm x 40,5mm	1093,5 mm2
motores	22mm x 28mm	616 mm2
rueda loca	33mm x 38 mm	1254 mm2
portapila triple	61mm x 77mm	4697 mm 2
batería 18650 litio	65mm x 18mm	1170 mm 2
TOTAL	—>	10679.5 mm2

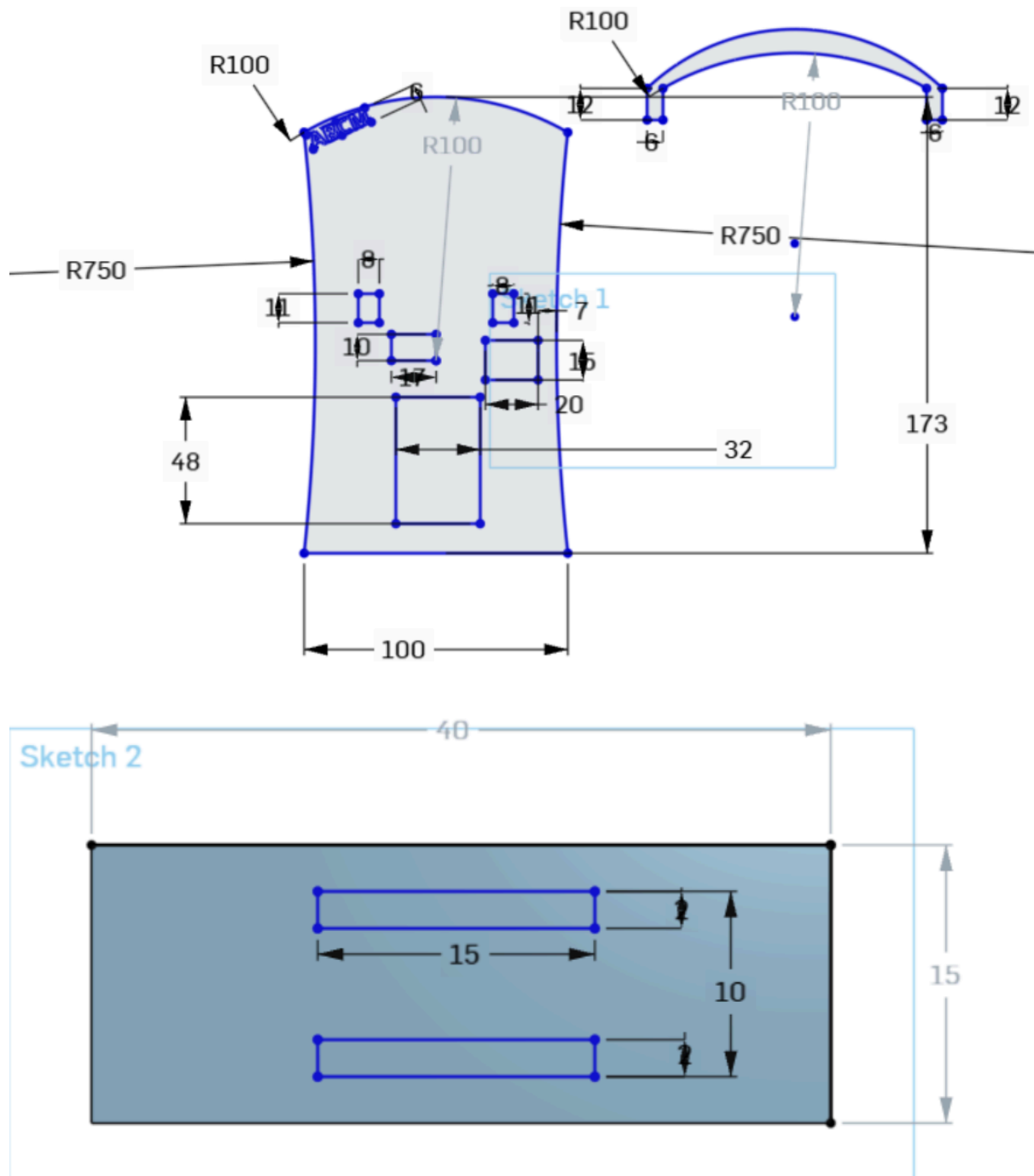
ANÁLISIS DE COSTOS:

Componente	Cantidad	Precio
Esp32 cam	1	\$13320
L298n Puente H	1	\$3395
Motorreductores	2	\$8044
Rueda loca	1	\$4700
Baterías litio	3	\$7032
Portabaterías	1	\$6800
TOTAL	—>	\$43291

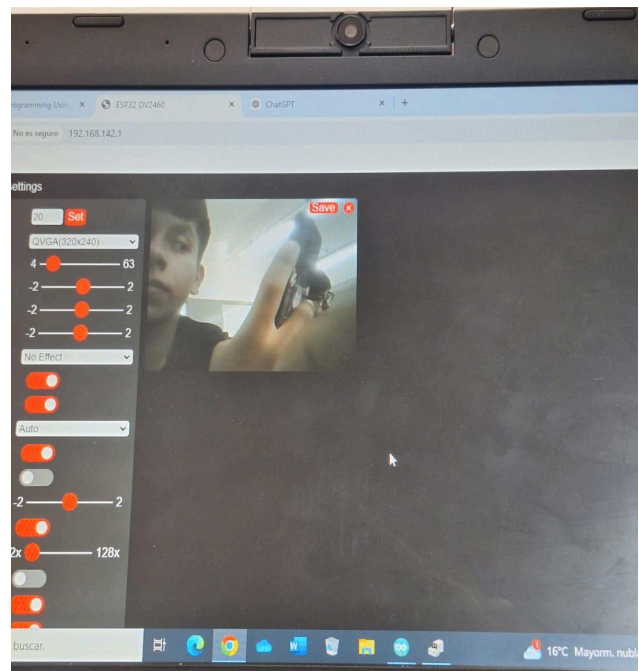
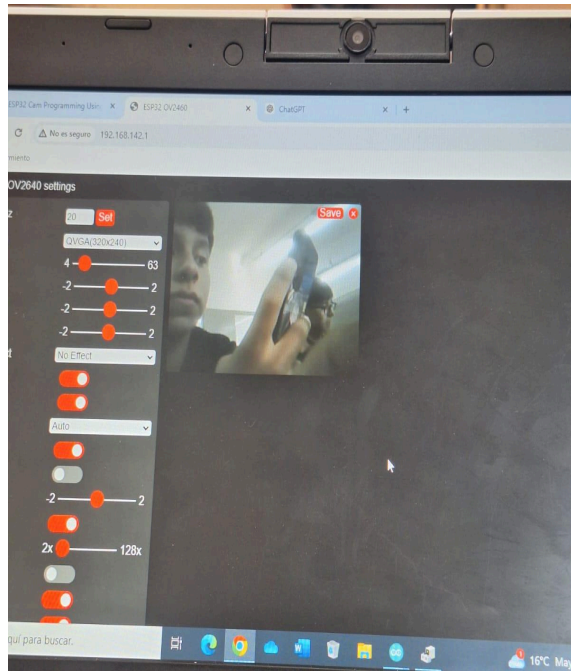
URL para descargar placa esp32 en arduino ide:

https://raw.githubusercontent.com/esp8266/Arduino/master/packages/esp8266/arduino-esp32/package_esp32_index.json

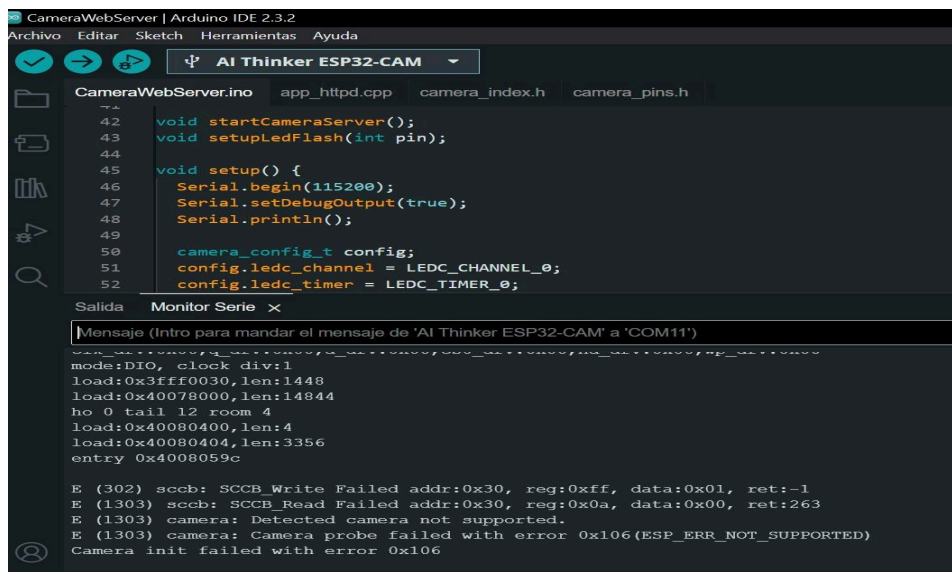
Gabinetes:



PRUEBAS DEL ESP 32 CAM:



Errores de la conexión:



Código:

```
#include "esp_camera.h"
#include <WiFi.h>
#include "esp_timer.h"
#include "img_converters.h"
#include "Arduino.h"
#include "fb_gfx.h"
#include "soc/soc.h" // Deshabilitar problemas de brownout
```

```

#include "soc/rtc_cntl_reg.h" // Deshabilitar problemas de brownout
#include "esp_http_server.h"

// Reemplaza con tus credenciales de red
const_char* ssid = "carlo";
const_char* password = "12345678";

#define PART_BOUNDARY "1234567890000000000000987654321"

#define CAMERA_MODEL_AI_THINKER
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

#define MOTOR_1_PIN_1 14
#define MOTOR_1_PIN_2 15
#define MOTOR_2_PIN_1 13
#define MOTOR_2_PIN_2 12

#define FLASH_LED_PIN 4
bool flashState = LOW;

static_const_char* _STREAM_CONTENT_TYPE = "multipart/x-mixed-replace;boundary="
PART_BOUNDARY;
static_const_char* _STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static_const_char* _STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n";

httpd_handle_t camera_httpd = NULL;
httpd_handle_t stream_httpd = NULL;

static_const_char PROGMEM INDEX_HTML[] = R"rawliteral(
<html>
<head>
<title>Auto Espia ABCM</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
h1 { color: white; }
body { font-family: Arial; text-align: center; margin: 0 auto; padding-top:
30px; background-color: black;}

```

```

    table { margin-left: auto; margin-right: auto; }
    td { padding: 8px; }
    .button {
        background-color: #2f4468;
        border: none;
        color: white;
        padding: 10px 20px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 18px;
        margin: 6px 3px;
        cursor: pointer;
        user-select: none;
        -webkit-tap-highlight-color: rgba(0, 0, 0, 0);
    }
    img { width: auto; max-width: 100%; height: auto; }
</style>
</head>
<body>
    <h1>Auto Espia ABCM</h1>
    <img src="" id="photo" alt="Video Stream">
    <table>
        <tr><td colspan="3" align="center"><button class="button"
onmousedown="toggleCheckbox('forward');"
ontouchstart="toggleCheckbox('forward');" onmouseup="toggleCheckbox('stop');"
ontouchend="toggleCheckbox('stop');">Adelante</button></td></tr>
        <tr>
            <td align="center"><button class="button"
onmousedown="toggleCheckbox('left');" ontouchstart="toggleCheckbox('left');"
onmouseup="toggleCheckbox('stop');"
ontouchend="toggleCheckbox('stop');">Izquierda</button></td>
            <td align="center"><button class="button"
onmousedown="toggleCheckbox('stop');"
ontouchstart="toggleCheckbox('stop');">Stop</button></td>
            <td align="center"><button class="button"
onmousedown="toggleCheckbox('right');" ontouchstart="toggleCheckbox('right');"
onmouseup="toggleCheckbox('stop');"
ontouchend="toggleCheckbox('stop');">Derecha</button></td>
        </tr>
        <tr><td colspan="3" align="center"><button class="button"
onmousedown="toggleCheckbox('backward');"
ontouchstart="toggleCheckbox('backward');" onmouseup="toggleCheckbox('stop');"
ontouchend="toggleCheckbox('stop');">Atras</button></td></tr>
        <tr><td colspan="3" align="center"><button class="button"
onmousedown="toggleCheckbox('flash');"
ontouchstart="toggleCheckbox('flash');">Luz</button></td></tr>
    </table>
    <script>
        function toggleCheckbox(action) {
            var xhr = new XMLHttpRequest();
            xhr.open("GET", "/action?go=" + action, true);
            xhr.send();
        }
    </script>

```



```

        xhr.onerror = function() {
            console.error("Request failed");
        };
    }

    window.onload = function() {
        document.getElementById("photo").src = window.location.origin +
":81/stream";
    };
</script>
</body>
</html>
)rawliteral";

static esp_err_t index_handler(httpd_req_t *req) {
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, (const char *)INDEX_HTML, strlen(INDEX_HTML));
}

static esp_err_t stream_handler(httpd_req_t *req) {
    camera_fb_t *fb = NULL;
    esp_err_t res = ESP_OK;
    size_t _jpg_buf_len = 0;
    uint8_t * _jpg_buf = NULL;
    char part_buf[64]; // Corrección: array de caracteres, no de punteros

    res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
    if (res != ESP_OK) {
        return res;
    }

    while (true) {
        fb = esp_camera_fb_get(); // Captura una imagen de la cámara
        if (!fb) {
            Serial.println("Camera capture failed");
            res = ESP_FAIL;
        } else {
            if (fb->width > 400) { // Comprueba si el tamaño de la imagen es
válido
                if (fb->format != PIXFORMAT_JPEG) {
                    // Convierte a JPEG si no está ya en ese formato
                    bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf,
&_jpg_buf_len);
                    esp_camera_fb_return(fb); // Libera el frame buffer
                    fb = NULL;
                    if (!jpeg_converted) {
                        Serial.println("JPEG compression failed");
                        res = ESP_FAIL;
                    }
                } else {
                    _jpg_buf_len = fb->len;

```



```

        _jpg_buf = fb->buf;
    }
}

if (res == ESP_OK) {
    // Envía la cabecera de la imagen
    size_t hlen = snprintf(part_buf, sizeof(part_buf), _STREAM_PART,
        _jpg_buf_len);
    res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
}

if (res == ESP_OK) {
    // Envía la imagen JPEG
    res = httpd_resp_send_chunk(req, (const char *)_jpg_buf,
        _jpg_buf_len);
}

if (res == ESP_OK) {
    // Envía la parte final del fragmento de transmisión
    res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
        strlen(_STREAM_BOUNDARY));
}

// Libera los recursos usados
if (fb) {
    esp_camera_fb_return(fb);
    fb = NULL;
    _jpg_buf = NULL;
} else if (_jpg_buf) {
    free(_jpg_buf);
    _jpg_buf = NULL;
}

if (res != ESP_OK) {
    break; // Sal del bucle si hay un error
}

return res; // Devuelve el resultado de la transmisión
}

// controlar desde la pagina
static esp_err_t cmd_handler(httpd_req_t *req){
    char* buf;
    size_t buf_len;
    char variable[32] = {0,};

    buf_len = httpd_req_get_url_query_len(req) + 1;
    if (buf_len > 1) {
        buf = (char*)malloc(buf_len);
    }

```

```
    if (!buf) {
        httpd_resp_send_500(req);
        return ESP_FAIL;
    }

    if (httpd_req_get_url_query_str(req, buf, buf_len) == ESP_OK) {
        if (httpd_query_key_value(buf, "go", variable, sizeof(variable)) !=
ESP_OK) {
            free(buf);
            httpd_resp_send_404(req);
            return ESP_FAIL;
        }
    } else {
        free(buf);
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }
    free(buf);
} else {
    httpd_resp_send_404(req);
    return ESP_FAIL;
}

// Control de motores y LED de flash
if (strcmp(variable, "forward") == 0) {
    Serial.println("Forward");
    digitalWrite(MOTOR_1_PIN_1, HIGH);
    digitalWrite(MOTOR_1_PIN_2, LOW);
    digitalWrite(MOTOR_2_PIN_1, HIGH);
    digitalWrite(MOTOR_2_PIN_2, LOW);
}
else if (strcmp(variable, "right") == 0) {
    Serial.println("Right");
    digitalWrite(MOTOR_1_PIN_1, LOW);
    digitalWrite(MOTOR_1_PIN_2, HIGH);
    digitalWrite(MOTOR_2_PIN_1, HIGH);
    digitalWrite(MOTOR_2_PIN_2, LOW);
}
else if (strcmp(variable, "left") == 0) {
    Serial.println("Left");
    digitalWrite(MOTOR_1_PIN_1, HIGH);
    digitalWrite(MOTOR_1_PIN_2, LOW);
    digitalWrite(MOTOR_2_PIN_1, LOW);
    digitalWrite(MOTOR_2_PIN_2, HIGH);
}
else if (strcmp(variable, "backward") == 0) {
    Serial.println("Backward");
    digitalWrite(MOTOR_1_PIN_1, LOW);
    digitalWrite(MOTOR_1_PIN_2, HIGH);
    digitalWrite(MOTOR_2_PIN_1, LOW);
    digitalWrite(MOTOR_2_PIN_2, HIGH);
}
else if (strcmp(variable, "stop") == 0) {
```

```

        Serial.println("Stop");
        digitalWrite(MOTOR_1_PIN_1, LOW);
        digitalWrite(MOTOR_1_PIN_2, LOW);
        digitalWrite(MOTOR_2_PIN_1, LOW);
        digitalWrite(MOTOR_2_PIN_2, LOW);
    } else if (strcmp(variable, "flash") == 0) {
        flashState = !flashState;
        digitalWrite(FLASH_LED_PIN, flashState);
    }
    else {
        httpd_resp_send_404(req); // Comando no encontrado
        return ESP_FAIL;
    }

    httpd_resp_sendstr(req, "OK"); // Comando procesado con éxito
    return ESP_OK;
}

void startCameraServer(){
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    config.server_port = 80;
    httpd_uri_t index_uri = {
        .uri          = "/",
        .method        = HTTP_GET,
        .handler        = index_handler,
        .user_ctx        = NULL
    };

    httpd_uri_t cmd_uri = {
        .uri          = "/action",
        .method        = HTTP_GET,
        .handler        = cmd_handler,
        .user_ctx        = NULL
    };

    httpd_uri_t stream_uri = {
        .uri          = "/stream",
        .method        = HTTP_GET,
        .handler        = stream_handler,
        .user_ctx        = NULL
    };

    if (httpd_start(&camera_httpd, &config) == ESP_OK) {
        httpd_register_uri_handler(camera_httpd, &index_uri);
        httpd_register_uri_handler(camera_httpd, &cmd_uri);
    }

    config.server_port += 1;
    config.ctrl_port += 1;
    if (httpd_start(&stream_httpd, &config) == ESP_OK) {
        httpd_register_uri_handler(stream_httpd, &stream_uri);
    }
}

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // Deshabilitar detector de brownout

    // Configuración de pines de motor y flash

```

```
pinMode(MOTOR_1_PIN_1, OUTPUT);
pinMode(MOTOR_1_PIN_2, OUTPUT);
pinMode(MOTOR_2_PIN_1, OUTPUT);
pinMode(MOTOR_2_PIN_2, OUTPUT);
pinMode(FLASH_LED_PIN, OUTPUT);
digitalWrite(FLASH_LED_PIN, flashState);

Serial.begin(115200);
Serial.setDebugOutput(false);

// Configuración de la cámara
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;

if(psramFound()){
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

// Inicialización de la cámara
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Fallo en la inicialización de la cámara con error 0x%x", err);
    ESP.restart(); // Reiniciar si la inicialización falla
    return;
}

// Conexión Wi-Fi
WiFi.begin(ssid, password);
```

```
Serial.println("Conectando a WiFi...");
unsigned long startAttemptTime = millis();

while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime < 10000) {
// 10 segundos de espera
    delay(500);
    Serial.print(".");
}

if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Error: No se pudo conectar a la red WiFi.");
    ESP.restart(); // Reiniciar si no se puede conectar
} else {
    Serial.println("");
    Serial.println("Conexión WiFi establecida.");
    Serial.print("Transmisión de cámara lista. Ve a: http://");
    Serial.println(WiFi.localIP());

    // Iniciar el servidor web de transmisión
    startCameraServer();
}
}

void loop() {
    // El bucle principal está vacío, ya que todo se maneja a través de
    interrupciones y solicitudes HTTP
}
```